OSLab

Skill-13

To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50

void execute_command(char *input)
{
   char *args[MAX_ARGS];
   int argc = 0;

   char *input_file = NULL;
   char *output_file = NULL;

   // Parse input
   char *token = strtok(input, " \t\n");

   while (token != NULL)
   {
      // Input redirection <
      if (strcmp(token, "<") == 0)
      {
         token = strtok(NULL, " \t\n");

         if (token == NULL)
         {
            printf("Error: missing input file\n");
            return;
         }
```

```
            input_file = token;
        }

        // Output redirection >
        else if (strcmp(token, ">") == 0)
        {
            token = strtok(NULL, " \t\n");

            if (token == NULL)
            {
                printf("Error: missing output file\n");
                return;
            }

            output_file = token;
        }

        // Normal command argument
        else
        {
            args[argc++] = token;
        }

        token = strtok(NULL, " \t\n");
    }

    args[argc] = NULL;

    if (argc == 0)
    {
        return;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
```

```
// ==========================
// INPUT REDIRECTION <
// ==========================

if (input_file != NULL)
{
   int fd = open(input_file, O_RDONLY);

   if (fd == -1)
   {
      perror("Input file");
      exit(EXIT_FAILURE);
   }

   // Redirect stdin to file
   if (dup2(fd, STDIN_FILENO) == -1)
   {
      perror("dup2");
      close(fd);
      exit(EXIT_FAILURE);
   }

   close(fd);
}

// ==========================
// OUTPUT REDIRECTION >
// ==========================

if (output_file != NULL)
{
   int fd = open(
      output_file,
      O_WRONLY | O_CREAT | O_TRUNC,
      0644
   );

   if (fd == -1)
   {
      perror("Output file");
      exit(EXIT_FAILURE);
   }

   // Redirect stdout to file
```

```
            if (dup2(fd, STDOUT_FILENO) == -1)
            {
                perror("dup2");
                close(fd);
                exit(EXIT_FAILURE);
            }

            close(fd);
        }

        // Execute command
        execvp(args[0], args);

        perror("execvp");
        exit(EXIT_FAILURE);
    }

    // Parent waits for child
    waitpid(pid, NULL, 0);
}

int main()
{
    char input[MAX_INPUT];

    while (1)
    {
        printf("myshell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
        {
            continue;
        }

        // Exit
        if (strcmp(input, "exit") == 0)
```

```
        {
            break;
        }

        execute_command(input);
    }

    return 0;
}
```

